AI Development Production Log

A Chronological Technical Record

VIDEO GAME CREATION

SLHE Game Programming - Video Game Creation

Purpose and Scope

This is a working, chronological record of how the course's content and its own production rulebook (Lesson_Production_Standards.docx) were built in collaboration with an AI assistant, kept at a more technical and granular level than the companion Case Study document. It exists to answer, in order, the question "what changed, and what specifically caused it" - useful for the eventual book's production-methodology chapter, and for anyone auditing why a given rule exists.

Reconstruction note: this log's entries through the point it was created are reconstructed from the project's existing paper trail (the Lesson Production Standards document, the Book Notes & To-Do scratchpad, and the project task list), not from a raw saved conversation. It could not be built as a true turn-by-turn transcript because the idea to preserve one came after a large part of the project's conversation had already occurred and aged out of recoverable context. From this point forward it is maintained live, entry by entry, as a genuine running log. See the Open Items section for a note on recovering more detail from the account-level data export.

Standards Rule Timeline

The project's production rulebook (Lesson_Production_Standards.docx) grew from one rule to fifteen over the course of building the first eleven of thirty lessons. The table below lists each rule in the order adopted, paired with the specific finding that produced it, pulled directly from the project's own notes.

Rule	Title	Finding That Produced It
1	Three Levels of Accomplishment	Established at project start as the base MVP / Complete / Refined structure for every weekly exercise.
2	Five Required Outputs Per Lesson	Established at project start: Handout, Teacher's Key, Slide Deck, Plan, and Prep Checklist for every lesson.
3	Every Lesson Is Self-Contained	Established early: no lesson should send a student

		hunting for an asset mid-class.
4	Visual Programming Prefers Purpose-Built Blocks	Adopted after finding generic visual-scripting blocks used where an engine actually offered a named, purpose-built equivalent.
5	Engine Deviations Require a Driving Reason	Written after discovering Unreal's content had drifted to flat-color Materials where every other engine used real sprite art, with no stated reason for the difference.
6	Beyond Level 3 - SuperStretch, Refinement, and Style	Added to formalize a standing SuperStretch tie-in plus a Refinement/Style pass and visual-scripting node-reuse guidance; scoped as forward-only for Lessons 1-15, not retrofitted.
7	Visual-Path Steps Are as Explicit as the Code Path	Written after GameMaker's GML Visual alt-path content was found to compress several real steps into vague prose while the code path spelled out every line, a gap found during hands-on testing.
8	Slide Decks Carry the Teacher's Script as Speaker Notes	Added once classroom delivery moved to Presenter View across two displays, making the notes pane a genuine live-teaching need rather than a nice-to-have.
9	Activity Diagrams and Engine Node Diagrams Are a Standing Requirement	Promoted from a one-lesson pilot (built because the same logic kept needing verbal re-explanation in class) to a full standing rule once the pilot's value was confirmed.
10	Lecture-Demo Engine Rotation	Written after discovering the live lecture demo had defaulted to the same single engine for twelve consecutive lessons, which a scan of the original schedule confirmed was never the actual plan.
11	Meaningful Variable Names Over Generic Counters	Adopted after a generic loop counter (i, e) was judged confusing for beginners compared to a descriptive name (enemy_count, new_enemy).
12	Common Ground Across Engines - Consistent Naming and Structure	Written after a real naming conflict in Unity (a loop counter that would have shadowed an existing field) was resolved, prompting the broader realization that students compare notes across engines and inconsistent names add a real translation cost.
13	Shared State Lives on a Dedicated Manager Object	Written after noticing Unity's code already routed shared state through a dedicated GameManager object while every other engine scattered the same state across whichever object needed it first - a correct pattern nobody had generalized.
14	Code Completeness by Level	Written after students were confused by Handouts/Keys showing only new or changed lines rather than complete, runnable methods.
15	Explicit, Beginner-Friendly Names	Written after a student misread Godot's WinLabel as referring to a window, prompting an audit of engine-jargon node names (Label, Node, Actor) across the project.

Notable Debugging Episodes

Entries below describe specific technical problems found and resolved during development, each with enough detail to stand alone as a troubleshooting or methodology anecdote.

The Spawn-Loop Self-Collision Bug

A hands-on test of an early lesson in Godot showed no enemies appearing on screen at all during play. Diagnosis traced the cause to the shared hit-detection architecture: it had no type-check on what triggered it, and at the original spacing, adjacent enemies' hitboxes overlapped at the moment of spawn and mutually destroyed each other before the first frame was ever rendered. Confirmed by reducing the enemy count and watching them survive. The fix applied was deliberately narrow - widen the spacing - rather than the deeper architectural fix (a real collision layer/mask system or a type-check), which was judged out of scope for the lesson's level. A full audit of the remaining engines found the same generic-trigger-fires-on-any-overlap pattern was independently already present (and separately already fixed) in one other engine, architecturally impossible in several engines by design, and present but unfixed in one more, which received the same spacing-based fix.

GameMaker Visual Targeting: Two Wrong Answers Before the Right One

An early GameMaker Visual (drag-and-drop) instruction for tinting a newly spawned enemy called for targeting a generic block with the new instance's reference - direct testing showed that block's targeting options didn't support it at all. The first fix (a different generic block with a typed field) worked and shipped, documented as a named exception to the 'prefer purpose-built blocks' rule. Later, further hands-on testing found a purpose-built scoping block could do the job correctly after all, if used in a specific nested arrangement not obvious from the block palette alone. Both the lesson content and the rule's documented exception were corrected a second time once this was found.

The Dropped Level 3 Sounds

A production convention requiring 'complete method' code listings (showing a whole updated method, not just new lines, so students always see runnable code) had a side effect: regenerating a later lesson's complete-method listings silently omitted previously-added, optional Level 3 sound-effect code (a win sound, a life-lost sound, a game-over sound) that students who'd done that earlier optional work would have added to their own project. Copying the newer listing into an existing project would overwrite and remove that code. This was invisible in the documents themselves and was only caught when a teacher noticed the symptom (games had gone quiet) rather than by inspecting the generated content directly. Fix: every affected complete-method listing across two lessons and four active engines had the sound lines restored, explicitly marked as carried over from the earlier lesson and non-highlighted (since they weren't new that week), with an added instruction telling a student who skipped the optional sound work to leave those lines out instead.

A Naming Mismatch Between a Handout and Its Key

A lesson's Teacher's Key referenced a UI element by one name (WaveAnnouncementText) while the paired student Handout's prose described the same feature without ever naming it, and a different, pre-existing UI element (WaveText, the persistent wave counter) existed nearby with a similar name. The ambiguity was resolved by explicitly naming the announcement element as its own, separate, clearly-labeled node in both documents, with the Key's setup instructions stating directly that it is a new node, separate from the existing counter.

Design Decisions Made Deliberately, Not by Default

- The Stalemate (a win/lose logic gap where some rounds end with no win, no loss, and no restart) was left in an early lesson on purpose as a find-and-fix exercise, then deliberately allowed to recur in a later lesson in a more conspicuous form, specifically so a student would be motivated to fix their own project rather than have the fix handed to them.
- A ternary-operator shorthand was pulled out of an early GameMaker code example, even though it was the instructor's own natural habit, because it was judged too advanced for students who hadn't yet met conditional expressions as values - deferred instead to a planned Semester 2 lesson, once students are extending a more mature codebase.
- A collision-handling pattern where one object (the Bullet) destroys both itself and the object it hit, rather than each object independently detecting and resolving its own collision, was deliberately kept as the simpler beginner pattern in an early lesson rather than 'corrected' mid-lesson, with the cleaner alternative explicitly saved for a later Refinement and Style discussion instead.
- A rotation for which engine is used in the live lecture demo each week was formally planned (Standards Rule 10) only after a review found the demo engine had defaulted to the same single engine for twelve lessons running - the fix was a deliberately narrowed rotation among only the engines with active students, rather than restoring an original, broader rotation that no longer matched how the lessons had actually been built.

Raw Conversation Recovered

The account-level data export (Settings > Privacy > Export) was checked and ruled out: it only covers claude.ai web-chat history, not Cowork desktop sessions, so this project's actual conversation never appeared in it. The real raw material turned up instead in the Claude desktop app's own local session cache, exported directly by Robert. It contained the complete, un-summarized record of this project - 391 real exchanges, first message to present, plus 39 marked points where the live conversation's context was compacted - and two short separate Claude Dispatch runs (remote sessions, generating and then correcting Lesson 6's sound effects).

That raw log has been machine-extracted into a structured, per-exchange archive (Book/Conversation_Archive/, documented in its own README) built for segment-by-segment review: each exchange carries the real human message, the real visible response, a brief record of which tools ran, and an empty commentary field meant to be filled in turn by turn in a future pass. This production log's entries above, through this point, were reconstructed from the project's secondary paper trail (the Standards doc, the Book Notes scratchpad, the task list) rather than from this archive - a next-pass opportunity is to cross-check and enrich each existing entry against the actual turns that produced it, now that they're recoverable.

Open Items

- Begin the segment-by-segment annotated walkthrough of Book/Conversation_Archive/main_session_conversation.jsonl - likely chaptered by its 39 compaction boundaries - filling in each turn's commentary field as it's reviewed.
- Cross-check this log's existing entries (and the Case Study's) against the newly-recovered raw turns for exact phrasing, precise sequencing, and any detail that didn't survive into the secondary sources this first pass was built from.
- Continue this log live, entry by entry, rather than reconstructing it retrospectively again - add a dated entry whenever a new Standards Rule is adopted, a notable bug is found and fixed, or a deliberate design decision (like the Stalemate) is made.
- Periodically revisit whether this production log and the companion narrative Case Study should be merged into a single document, or kept separate for different audiences (production/technical readers versus a general education audience).